

Relatório de Desenvolvimento Avançado de Aplicações para Internet

Projetos

Feito por:

João Matias 81983

Índice

Índice	2
Introdução:.....	3
Framework Laravel:	4
Desenvolvimento:.....	5
Utilizadores:.....	5
Projetos:.....	6
Dashboard:	6
Fundamentos para o Desenvolvimento:	7
Variáveis de Ambiente	7
Filesystem e Gestão de Ficheiros	7
Sistema de Rotas	7
Base de Dados e Escalabilidade.....	7
Conclusão:	8
Instalação:	9
Requisitos:	9
Instalar Composer:	9
Instalação Node.js:	10
Passo a passo:.....	11
Anexos:.....	12

Introdução:

Este projeto dá continuidade ao desenvolvimento de uma plataforma de gestão destinada a investigadores e técnicos de apoio, inserida no contexto escolar. O sistema tem como principal objetivo disponibilizar um controlo centralizado de projetos, bem como a gestão dos respetivos anexos que permite uma organização mais eficiente da informação associada à investigação.

Para além disso, a plataforma integra funcionalidades avançadas de gestão de equipas que possibilita a definição de perfis, permissões e a aplicação de regras de negócio específicas, ajustadas às necessidades do projeto. Estas regras visam assegurar a coerência dos processos, bem como o cumprimento dos requisitos próprios para não existir falta de informação.

A aplicação foi desenvolvida com recurso à framework **Laravel**, baseada na linguagem **PHP**, tirando partido da sua arquitetura robusta e das boas práticas que promove no desenvolvimento de aplicações web. Ao longo deste documento será analisada a estrutura da aplicação, o seu funcionamento e as principais decisões técnicas adotadas durante o processo de desenvolvimento.

No final, também haverá um guia de como instalar a partir do GitHub ou do início instalando tudo desde o Composer e o Node.JS, também como a instalação e criação da base de dados e da estrutura do Laravel.

Framework Laravel:

O **Laravel** é uma framework de desenvolvimento web em **PHP** que possui uma estrutura própria e segue a arquitetura **MVC (Model–View–Controller)**, permitindo uma organização clara e eficiente do código.

Na arquitetura MVC, o **Model** é responsável pela gestão da lógica de dados e pela comunicação com a base de dados. No Laravel, esta camada é implementada através do **Eloquent ORM**, que facilita a interação com a base de dados de forma simples, intuitiva e organizada. O Eloquent permite criar queries de forma fácil e rápida, utilizando objetos em vez de SQL puro, o que torna o código mais legível e fácil de manutenção.

A **View** corresponde à camada de apresentação, ou seja, tudo o que é mostrado ao utilizador. O Laravel utiliza o Blade, que permite misturar HTML com código PHP de forma limpa e estruturada. Isto possibilita uma separação clara da lógica e tornando o código mais organizado e fácil de compreender.

O **Controller** é responsável por aplicar as regras de negócio da aplicação. É nesta camada que se processam os dados recebidos, se comunica com os Models e se decide qual a View a apresentar. Cada módulo da aplicação tem o seu próprio controller, contribuindo para uma aplicação robusta, bem estruturada e por módulos.

Relativamente à segurança, o Laravel oferece proteção nativa contra **SQL Injection** e **CSRF (Cross-Site Request Forgery)**, o que aumenta significativamente a segurança da aplicação. Além disso, a sua escalabilidade é uma das suas grandes vantagens, pois a adição de novas funcionalidades ou regras de negócio não compromete a organização do código.

No que diz respeito aos URLs, o Laravel utiliza um sistema de rotas. O utilizador não tem conhecimento direto do ficheiro PHP que está a ser executado, o que aumenta a segurança da aplicação. Cada rota aponta para um controller específico, onde é definida a lógica e a view a apresentar. As rotas podem ainda utilizar middlewares, que funcionam como uma camada adicional de segurança, permitindo controlar o acesso às rotas com base em permissões, autenticação ou outros critérios.

Em resumo, o Laravel é uma framework fácil de aprender, segura e bem estruturada, ideal para o desenvolvimento de aplicações web modernas, organizadas e de fácil manutenção.

Desenvolvimento:

Durante o desenvolvimento deste trabalho foi necessária a integração de vários pacotes adicionais, com o objetivo de acrescentar funcionalidades relevantes à aplicação. Um dos pacotes utilizados foi o **PHPMailer**, responsável pelo envio de correio eletrónico, permitindo o envio de mensagens através de uma conta Gmail, bastando para isso configurar o respetivo acesso.

Outro pacote integrado foi o **Livewire**, que apresenta uma excelente integração com o **Laravel**, possibilitando a criação de interfaces mais dinâmicas. A utilização deste pacote elimina a necessidade de recarregar a página para a execução da maioria das ações, melhorando significativamente a experiência do utilizador.

Para a componente visual da aplicação foi utilizado o **Bootstrap UI**, garantindo uma interface mais consistente e responsiva. Todos estes pacotes foram instalados através do **Composer**, um gestor de dependências para PHP. Adicionalmente, recorreu-se ao **Node.js** para a instalação e gestão do Bootstrap, assegurando a correta compilação dos recursos visuais.

Utilizadores:

O desenvolvimento do módulo de utilizadores foi realizado de acordo com as orientações do professor, tendo sido definidos dois tipos de utilizador com permissões distintas. Cada tipo de utilizador está associado a módulos específicos do sistema, garantindo um controlo adequado das funcionalidades disponíveis a cada perfil.

Adicionalmente, foi implementado um estado para os utilizadores, permitindo identificar se estes se encontram ativos ou inativos. Desta forma, os utilizadores que não estejam ativos ficam automaticamente impedidos de aceder ao sistema, reforçando a segurança e o controlo de acessos da aplicação.

Projetos:

O módulo de projetos constitui a funcionalidade mais importante do sistema, uma vez que a gestão dos projetos é o seu elemento central. Através deste módulo, um investigador pode propor um novo projeto, sendo necessário, numa primeira fase, o preenchimento das informações iniciais. Apenas após a submissão desses dados é possível adicionar os anexos associados ao projeto e criar a equipa responsável, tendo em conta que um projeto pode envolver mais do que um investigador.

Quando um projeto é inicialmente proposto, este assume o estado **Draft**, permitindo ao investigador rever e alterar informações ou anexos sempre que esteja neste estado. Após essa validação, o projeto pode ser submetido para autorização. Apenas neste estado os Técnicos de Apoio conseguem visualizar e ficar responsáveis pelo projeto, não sendo permitida qualquer modificação dos seus dados, apenas a adição de comentários e a comunicação com outros técnicos.

Depois de verificado, o projeto passa para o estado **Autorizado**. A partir deste momento, o projeto pode ser iniciado ou cancelado. Caso seja iniciado, poderá posteriormente assumir os estados de **Terminado** ou **Reprovado**, correspondendo esta à sequência completa de estados que um projeto pode ficar.

Dashboard:

Após o login no sistema, cada utilizador é direcionado para um dashboard personalizado, adaptado ao seu perfil e às suas responsabilidades dentro da plataforma. O objetivo é garantir que cada tipo de utilizador tenha acesso rápido às informações mais relevantes para o seu trabalho.

Dashboard do Investigador

O dashboard do Investigador está orientado para a gestão e acompanhamento dos seus próprios projetos. Este dashboard foca-se essencialmente na submissão, consulta e acompanhamento dos projetos, permitindo ao Investigador ter uma visão clara do progresso das suas solicitações.

Dashboard do Técnico de Apoio

O dashboard do Técnico de Apoio é mais operacional e centrado na gestão e acompanhamento dos projetos atribuídos. Este dashboard foi concebido para facilitar a gestão do fluxo de trabalho, permitindo ao Técnico de Apoio acompanhar e intervir nos processos de forma eficiente.

Fundamentos para o Desenvolvimento:

Durante o desenvolvimento deste trabalho, o Laravel revelou-se uma framework extremamente sólida, fornecendo diversos fundamentos que contribuíram para a organização, segurança e escalabilidade da aplicação.

Variáveis de Ambiente

Um dos aspetos mais relevantes foi a utilização de variáveis de ambiente. Em vez de expor diretamente identificadores (IDs) ou valores sensíveis no código, o Laravel permite definir constantes e configurações no ficheiro .env. Desta forma, as regras de negócio tornam-se mais claras e legíveis, uma vez que são utilizadas variáveis com nomes representativos em vez de valores numéricos “hardcoded”. Além disso, esta abordagem facilita significativamente a manutenção do sistema, permitindo alterar configurações sem necessidade de modificar diretamente o código-fonte.

Filesystem e Gestão de Ficheiros

Outro fundamento importante do Laravel é o sistema de filesystem, que permite gerir ficheiros de forma simples, segura e eficiente. O Laravel suporta diferentes drivers (local, FTP, S3, entre outros), possibilitando o envio de ficheiros para diversas plataformas do mercado com grande flexibilidade. Comparativamente ao PHP tradicional, onde ao enviar um ficheiro por FTP o caminho poderia ficar exposto, o Laravel permite controlar o acesso aos ficheiros através de rotas protegidas. Isto significa que apenas utilizadores autenticados podem aceder aos recursos, reforçando assim a segurança da aplicação.

Sistema de Rotas

O sistema de rotas do Laravel é outro dos seus grandes pontos fortes. A definição clara e organizada das rotas torna a aplicação mais estruturada e fácil de compreender. Cada rota está associada a um Controller específico, responsável por tratar a respetiva lógica. Esta separação de responsabilidades melhora a organização do projeto e facilita a manutenção e evolução do sistema. Mesmo em aplicações com centenas ou milhares de rotas, o Laravel mantém uma estrutura clara e organizada, permitindo uma gestão eficiente do fluxo da aplicação.

Base de Dados e Escalabilidade

O Laravel também simplifica significativamente a gestão da base de dados. Através das migrations é possível criar, alterar e testar estruturas de base de dados de forma rápida e controlada. Este mecanismo facilita o desenvolvimento, a realização de testes e a futura escalabilidade da aplicação, garantindo que a base de dados pode evoluir de forma estruturada e consistente.

Conclusão:

O desenvolvimento deste projeto permitiu-me consolidar conhecimentos técnicos na área do desenvolvimento web, recorrendo à **framework** Laravel como base estrutural da aplicação. Ao longo do trabalho, foi possível implementar uma solução organizada, segura e escalável, adaptada às necessidades dos diferentes tipos de utilizadores do sistema. Do ponto de vista técnico, o Laravel revelou-se uma ferramenta robusta e adequada ao desenvolvimento do projeto. A utilização de variáveis de ambiente permitiu tornar o código mais claro, organizado e de fácil manutenção, evitando a exposição direta de valores sensíveis. O sistema de rotas contribuiu para uma estrutura bem definida e intuitiva, facilitando a separação de responsabilidades e a organização da lógica da aplicação.

Em conclusão, a realização deste trabalho revelou-se fundamental para consolidar os meus conhecimentos em **Laravel**, permitindo-me aprofundar a compreensão da sua estrutura, funcionamento e boas práticas de desenvolvimento. Para além disso, possibilitou a integração e exploração de novos pacotes e ferramentas que até então não conhecia, contribuindo para o enriquecimento das minhas competências técnicas. Este processo permitiu-me também compreender de forma mais clara e estruturada os requisitos definidos no início do projeto, bem como a melhor forma de os implementar de acordo com as necessidades propostas.

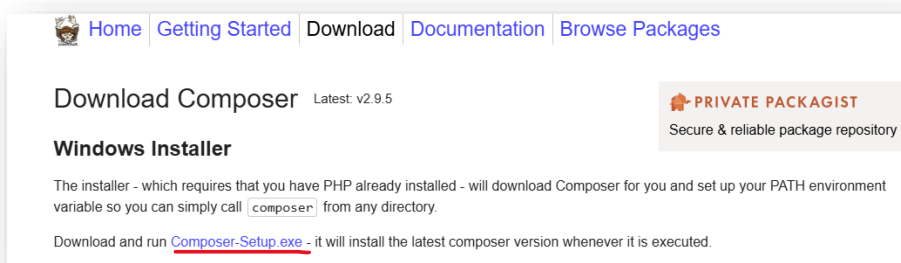
Instalação:

Requisitos:

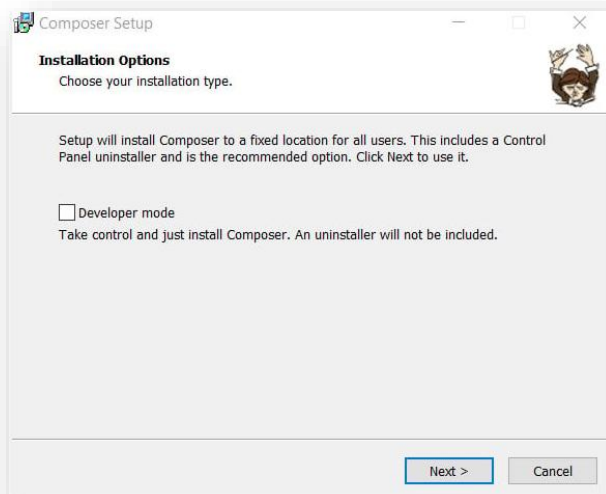
Para a correta execução da aplicação é necessário ter instalado o PHP na máquina, preferencialmente na versão 8.0 ou superior recomendada. É igualmente necessário dispor do Composer, para a gestão de dependências do projeto, e do Node.js, para a instalação e compilação dos recursos associados à componente visual. Adicionalmente, deve estar configurado um motor de base de dados, como por exemplo o MySQL, podendo ser utilizada uma ferramenta de administração como o DBeaver para a gestão da base de dados.

Instalar Composer:

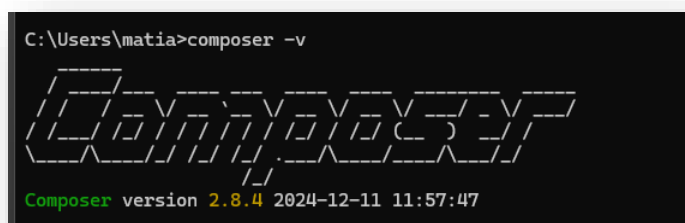
<https://getcomposer.org/download/>



Download o Composer-Setup.exe, depois execute:



Após a instalação para verificar se está corretamente instalado:



Instalação Node.Js:

<https://nodejs.org/en/download>

The screenshot shows the 'Download Node.js®' page. At the top, there are dropdown menus for 'v24.13.1 (LTS)', 'Windows', 'Docker', and 'npm'. Below this is a green banner with an 'Info' icon and text: 'Want new features sooner? Get the latest Node.js version instead and try the latest improvements!'. The main content area has a dark background with white text showing Docker installation instructions:

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:24-alpine
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.13.1".
12
13 # Verify npm version:
14 npm -v # Should print "11.8.0".
```

Below the code is a 'PowerShell' label and a 'Copy to clipboard' button. Underneath is a note: 'Docker is a containerization platform. If you encounter any issues please visit Docker's website'. At the bottom, there are two buttons: 'Windows Installer (.msi)' (highlighted with a red box) and 'Standalone Binary (.zip)'.

Ao executar o instalador do Node.js, deverá clicar sempre em “Next” até concluir a instalação. Após terminar, é aconselhável verificar se a instalação foi realizada com sucesso, abrindo a linha de comandos e executando o comando `node -v`, que deverá apresentar a versão instalada. Deve ainda confirmar se a variável de ambiente foi corretamente configurada, garantindo que o sistema reconhece o comando `node`.

```
C:\Users\matia>node -v
v22.16.0
```

Passo a passo:

1. Após descompactar a pasta, verifique o ficheiro(.env.example) em que você vai criar um ficheiro .env e copia o conteúdo para o ficheiro que acabou de criar onde, vai mudar para as suas credenciais onde nos locais onde estiverem vazios.
2. Agora irá ao seu motor de base de dados, MySQL onde vai criar apenas a base de dados onde guardará o nome, para colocar no passo anterior.
3. De seguida na linha de comandos na pasta raiz do projeto vai rodar o **composer install** e depois ainda na raiz do projeto vai fazer o **npm install**.
4. De seguida ainda na pasta raiz do projeto irá rodar o comando **php artisan key:generate** que vai gerar uma chave única para o seu projeto.
5. De seguida vai criar a base de dados rodando o comando **php artisan migrate** e assim vai criar a base de dados.
6. Agora nesta fase vai rodar o comando **npm run build** para os assets serem atualizados na pasta build para o programa rodar sem alguma deficiência.
7. Agora para ver o programa a funcionar rode o **php artisan serve** e agora pode usufruir do sistema a rodar 100%.

Anexos:

- <https://laravel.com/>
- <https://livewire.laravel.com/>
- <https://laravel.com/docs/12.x/mail>
- <https://www.webappfix.com/post/how-to-send-mail-using-phpmailer-in-laravel.html>
- <https://laracasts.com/>
- <https://raviyatechnical.medium.com/laravel-12-bootstrap-5-auth-scaffolding-tutorial-with-laravel-ui-package-9ec9c640ffdc>